

Department of Computer Science and Engineering

Indian Institute of Technology Guwahati

Non-Graded Practice Session

Debugging Luit with GDB

Traps, System Calls, Page Tables, and Process State

CS3106L - Operating Systems: Concepts and Construction

BrahmaputraOS / kernel *Luit* - Student Release V3.0

Guided laboratory practice handout

Dr. Satyajit Das

Department of Computer Science and Engineering, IIT Guwahati

Contents

1 Purpose of This Practice Session	2
1.1 Learning outcomes	2
1.2 What this session is not	2
1.3 Suggested working plan	3
2 The Release You Will Use	3
2.1 Know which terminal is which	3
3 Start Luit Under GDB	3
3.1 Terminal 1: start QEMU paused	4
3.2 Terminal 2: attach GDB	4
4 Practice A - Find the Scheduler	4
5 Practice B - Follow a System Call	5
5.1 Stop at the dispatch point	5
5.2 Prove that it came from a user <code>ecall</code>	6
5.3 Watch the return value appear	7
6 Practice C - Which Page Table Is Active?	7
7 Practice D - Walk an Sv39 Page Table	8
7.1 First compute the three Sv39 indices	8
7.2 Let Luit's own <code>walk()</code> find the leaf PTE	8
7.3 Convert the leaf PTE into a physical page address	9
7.4 Optional challenge: prove kernel isolation in the same page table	9
8 Practice E - Inspect the Process Table	10
8.1 Inspect the state prepared for later labs	10
9 Practice F - Locate the Future Hook Points	11
9.1 Lab 2 hook: <code>trace_record()</code>	11
9.2 Lab 3 hook: <code>usysinfo_update()</code>	11
10 Final Concept Check	12
11 Clean Exit and Troubleshooting	12
11.1 Clean exit	12
11.2 Common problems	13
12 One-Page GDB Quick Reference	13

1 Purpose of This Practice Session

This session connects the mechanisms discussed in class to a running Luit kernel. You will use QEMU and GDB to stop the machine at carefully chosen points and answer four questions from live state:

How did we enter the kernel? → Which system call is this? → Which page table is active? → Which process owns this state?

The aim is not to memorise GDB commands. The aim is to become comfortable locating evidence inside a real kernel. These are the same debugging skills you will need when you later implement the kernel tracing and shared-information-page labs.

1.1 Learning outcomes

After completing the session, you should be able to:

1. start Luit under QEMU's GDB remote stub and attach a RISC-V-aware GDB;
2. stop at `scheduler()`, `usertrap()`, and `syscall()` and read the current call stack;
3. explain how a user `ecall` becomes `scause == 8` and reaches `syscall()`;
4. locate a system-call number in the saved `a7`, its first argument in saved `a0`, and its return value in saved `a0`;
5. compare `satp` with a process's `pagetable` and explain why Luit does not need a page-table switch merely to enter the kernel;
6. walk an Sv39 page table for a chosen virtual address and decode V/R/W/X/U permission bits;
7. prove that the shared information page is user-readable but user-read-only;
8. inspect the process table and the Lab 2/Lab 3 state already present in `struct proc`; and
9. distinguish the working starter infrastructure from code that is intentionally still a later-lab TODO.

1.2 What this session is not

- It is **not graded**; there is no submission and no marks for speed.
- It is **not** Lab 2 or Lab 3. Do not fill the TODOs in `kernel/trace.c` or `kernel/usysinfo.c` today.
- It is not enough to copy the commands. At each stop, identify what the values mean.
- It is not a Linux debugging exercise. GDB runs on the Linux host, but the code being debugged is the RISC-V Luit kernel inside QEMU.

1.3 Suggested working plan

Phase	Activity	Time
A	Verify the release and establish the two-terminal workflow	15 min
B	Stop in the scheduler and orient yourself in kernel state	20 min
C	Follow a user system call across the trap boundary	40 min
D	Inspect <code>satp</code> , walk the Sv39 page table, and decode permissions	45 min
E	Inspect the process table and the Lab 2/Lab 3 hook points	20 min
F	Complete the final concept check and cleanly exit GDB/QEMU	10 min

2 The Release You Will Use

The supplied package contains the complete Luit teaching base plus the specifications for later Labs 2 and 3. For this practice session, work only inside the `luit/` directory.

LINUX HOST TERMINAL - run outside QEMU

```
cd LUIT_Labs2-3_Student/luit
pwd
ls kernel user hal tests
ls kernel.elf fs.img 2>/dev/null || true
```

Build the starter:

LINUX HOST TERMINAL - run outside QEMU

```
make
```

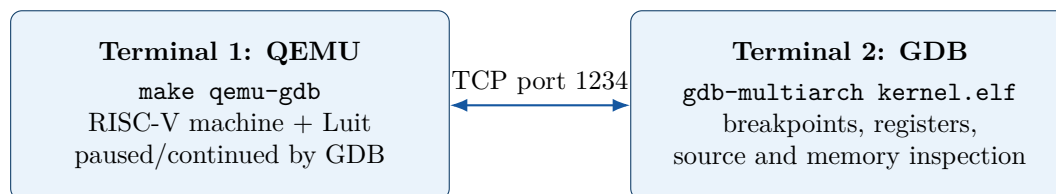
A successful build is enough for today's session. You do not need the Lab 2 or Lab 3 graders.

The later-lab public tests are not baseline checks

The V3.0 release intentionally leaves the Lab 2 recorder in `kernel/trace.c` and the Lab 3 seqlock writer in `kernel/usysinfo.c` unfinished. Therefore, do **not** interpret a failing `make grade LAB=2` or `make grade LAB=3` as a setup failure before you have implemented those labs.

2.1 Know which terminal is which

You will keep two host terminals open throughout the session:



When QEMU is running normally, Terminal 1 also becomes the Luit serial console and eventually displays the `luit$` prompt. When the debugger stops the virtual CPU, the guest console freezes because the emulated CPU is frozen.

3 Start Luit Under GDB

3.1 Terminal 1: start QEMU paused

From the `luit/` directory:

LINUX HOST TERMINAL - run outside QEMU

```
make qemu-gdb
```

The Makefile starts QEMU with `-S` and `-gdb tcp::1234`. The machine is deliberately paused before it executes normally.

3.2 Terminal 2: attach GDB

From the same `luit/` directory:

LINUX HOST TERMINAL - run outside QEMU

```
gdb-multiarch kernel.elf
```

If the shipped `.gdbinit` is loaded, GDB prints a message similar to:

EXPECTED SHAPE / OBSERVATION - do not type this block

```
[Luit] connected. Helpers: trapregs, procs, tf <ptr>.
[Luit] Suggested first breakpoints: b usertrap / b syscall / b scheduler
```

Try the helper help entries:

GDB TERMINAL - commands entered at the (gdb) prompt

```
help trapregs
help procs
help tf
```

If GDB refuses to auto-load .gdbinit

On the department machines the safe path should already be configured. On your own machine, add the following to `~/.config/gdb/gdbinit`, replacing the path with the absolute path to this checkout:

```
add-auto-load-safe-path /absolute/path/to/luit/.gdbinit
```

Then restart GDB. No `sudo` access is required.

One small detail about the shipped procs helper

The current V3.0 `.gdbinit` helper scans process-table slots 0 through 15 for a compact display. The kernel itself has `NPROC = 64`. Do not conclude that Luit has only 16 process slots merely because the helper is short.

Checkpoint: Debugger attached

Before continuing, you should have QEMU waiting in Terminal 1 and a live `(gdb)` prompt in Terminal 2. If GDB says port 1234 is already in use or cannot connect, check whether an older QEMU process from your account is still running.

4 Practice A - Find the Scheduler

The scheduler is a good first breakpoint because it proves that the kernel completed boot setup and reached the loop that chooses runnable processes.

GDB TERMINAL - commands entered at the (gdb) prompt

```
b scheduler
c
```

When GDB stops:

GDB TERMINAL - commands entered at the (gdb) prompt

```
bt
list
p/x $satp
p/x kernel_pagetable
```

Do not worry about the exact numeric addresses. Identify the function and the source file. Then inspect the process table:

GDB TERMINAL - commands entered at the (gdb) prompt

```
procs
```

At the first scheduler stop you may see only the earliest process state, because boot and scheduling are still progressing.

Practice A1 - Orient yourself

Write down:

1. the source file containing `scheduler()`;
2. the current `satp` value; and
3. which process slots are currently visible through `procs`.

Then continue once or twice and observe how the visible process state changes.

Remove the scheduler breakpoint so it does not stop you continuously:

GDB TERMINAL - commands entered at the (gdb) prompt

```
info breakpoints
disable 1
```

If your scheduler breakpoint is not breakpoint 1, use the number shown by `info breakpoints`.

5 Practice B - Follow a System Call

Every Luit system call passes through `syscall()` in `kernel/syscall.c`. The syscall number is saved in the user trapframe's `a7`; the first argument begins in saved `a0`; the return value is written back to saved `a0`.

5.1 Stop at the dispatch point

Set the breakpoint and continue:

GDB TERMINAL - commands entered at the (gdb) prompt

```
b syscall
c
```

A breakpoint on a C function normally stops at its first executable source line. In this function, first step over the assignment that obtains the current process so that the local variable `p` is valid:

GDB TERMINAL - commands entered at the (gdb) prompt

```
next
p p->pid
p p->name
p p->tf->a7
p/x p->tf->a0
bt
```

On the Linux host, cross-check the number against the ABI table:

LINUX HOST TERMINAL - run outside QEMU

```
cat kernel/syscall.tbl
```

Common early numbers include `fork=1`, `read=5`, `exec=7`, `getpid=11`, and `write=16`. Always trust the table in your release rather than a number remembered from another operating system.

Record: B1 - One live system call

Record one call you observed:

Process name / PID

Saved `a7`

System-call name

Saved `a0` before dis-
patch

5.2 Prove that it came from a user `ecall`

While stopped in `syscall()`, run:

GDB TERMINAL - commands entered at the (gdb) prompt

```
trapregs
p/x p->tf->epc
p/x $sepc
```

For a direct user system-call trap, `scause` is 8. Notice the relationship between the saved CSR `sepc` and `p->tf->epc`.

Why two program-counter values can differ by four

On an `ecall`, `usertrap()` first saves `sepc` into the trapframe and then advances the trapframe's `epc` by 4 before calling `syscall()`. The process must return to the instruction *after* the `ecall`; otherwise it would execute the same system call again.

Practice B2 - Explain the boundary

Using the values you saw, answer these in one or two sentences each:

1. Why do we inspect `p->tf->a7` instead of assuming the live kernel register `$a7` still contains the syscall number?
2. Why is `p->tf->epc` advanced before `syscall()` runs?
3. What does `scause == 8` mean on RISC-V?

5.3 Watch the return value appear

Use `list` to find the dispatch statement:

GDB TERMINAL - commands entered at the (gdb) prompt

```
list
```

Step until you are immediately before the line that calls `syscalls[num]()` and then use `next` once to execute the handler. After it returns:

GDB TERMINAL - commands entered at the (gdb) prompt

```
p p->tf->a0
```

That is the value that will eventually be restored into user register `a0`.

Practice B3 - Observe several calls

Continue to the next syscall breakpoint several times. For at least three stops, record the process name, saved `a7`, syscall name, and saved `a0`. Look for one read/write-style call and one process-management call.

6 Practice C - Which Page Table Is Active?

This exercise connects the page-table lecture directly to Luit's design. Luit maps the kernel into every process address space, with kernel mappings lacking `PTE_U`. Therefore, when a user process traps into the kernel, Luit can keep that process page table active during the trap handler.

Stop again in `syscall()` and step once so `p` is valid:

GDB TERMINAL - commands entered at the (gdb) prompt

```
c
next
p/x p->pagetable
p/x $satp
```

The low 44 bits of `satp` contain the root page-table physical page number (PPN). Compare it with the process page-table pointer shifted by 12:

GDB TERMINAL - commands entered at the (gdb) prompt

```
set $satp_ppn = $satp & 0xfffffffffff
set $proc_ppn = ((unsigned long)p->pagetable) >> 12
p/x $satp_ppn
p/x $proc_ppn
```

They should match while this process is executing in the kernel.

What this proves about Luit

A system call does **not** require Luit to replace `satp` at trap entry. The current process page table already contains the kernel's high mappings, so supervisor code can execute immediately. User mode still cannot access those mappings because their PTEs do not contain `PTE_U`.

When the process later gives up the CPU and the scheduler is running between processes, `scheduler()` switches back to `kernel_pagetable`. Before a process resumes, it loads that process's page-table root again.

Practice C1 - The key page-table question

If process A and process B have different `pagetable` root addresses, how can both roots still lead to the same kernel code at physical address `0x80200000`? Use the phrase *borrowed/shared upper-level kernel mappings* in your explanation.

7 Practice D - Walk an Sv39 Page Table

Lab 3 reserves one user virtual page at `USYSINFO_VA = 0x7FFF000`. The mapping infrastructure is already present in the V3.0 starter, so you can inspect the PTE even before the later seqlock writer is implemented.

7.1 First compute the three Sv39 indices

A 4 KiB page gives a 12-bit offset. Sv39 then uses three 9-bit indices. Let GDB perform the bit extraction:

GDB TERMINAL - commands entered at the (gdb) prompt

```
p/x (0x7ffff000 >> 30) & 0x1ff
p/x (0x7ffff000 >> 21) & 0x1ff
p/x (0x7ffff000 >> 12) & 0x1ff
```

Record the three indices as `VPN[2]`, `VPN[1]`, and `VPN[0]`.

7.2 Let Luit's own `walk()` find the leaf PTE

While stopped in a process context at `syscall()`:

GDB TERMINAL - commands entered at the (gdb) prompt

```
set $ptep = walk(p->pagetable, 0x7ffff000, 0)
p/x $ptep
set $pte = *(unsigned long *)$ptep
p/x $pte
```

Now test the permission bits individually:

GDB TERMINAL - commands entered at the (gdb) prompt

```
p ($pte & 0x1) != 0
p ($pte & 0x2) != 0
p ($pte & 0x4) != 0
p ($pte & 0x8) != 0
p ($pte & 0x10) != 0
```

Interpret them in this order:

Bit	Mask	Name	Question
0	0x1	V	Is the mapping valid?
1	0x2	R	May it be read?
2	0x4	W	May it be written?
3	0x8	X	May instructions execute from it?
4	0x10	U	May user mode access it?

For the information page, the important result is:

EXPECTED SHAPE / OBSERVATION - do not type this block

```
V = 1, R = 1, W = 0, U = 1
```

The exact raw PTE value can differ if hardware-maintained status bits are set; judge the permissions by the individual bit tests.

Practice D1 - Explain the safety property

Why is the combination R=1, W=0, U=1 exactly what a kernel-to-user information page needs? What failure would become possible if W=1?

7.3 Convert the leaf PTE into a physical page address

The PPN starts at PTE bit 10. Convert it back to a page-aligned physical address:

GDB TERMINAL - commands entered at the (gdb) prompt

```
p/x (($pte >> 10) << 12)
```

This is the physical page backing the fixed user virtual address 0x7FFFF000 for the current process.

Record: D2 - Translation record

Fill in one complete translation:

Virtual address	0x7FFFF000
VPN[2],	VPN[1],
VPN[0]	
Leaf PTE address	
Raw leaf PTE	
Decoded V/R/W/X/U	
Physical page address	

7.4 Optional challenge: prove kernel isolation in the same page table

The current process page table also contains the Luit kernel text at 0x80200000. Walk that address:

GDB TERMINAL - commands entered at the (gdb) prompt

```
set $kptep = walk(p->pagetable, 0x80200000, 0)
set $kpte = *(unsigned long *)$kptep
p/x $kpte
p ($kpte & 0x2) != 0
p ($kpte & 0x4) != 0
p ($kpte & 0x8) != 0
p ($kpte & 0x10) != 0
```

You should find readable/executable kernel text, but PTE_U must be clear. This is the answer to a subtle question: *a virtual address can exist in the current page table and still be inaccessible from user mode.*

Practice D3 - Compare two mappings

Compare the info-page PTE with the kernel-text PTE. Both are present in the same process page table. Why can user code read the first but not the second?

8 Practice E - Inspect the Process Table

Return to a `syscall()` breakpoint and make sure `p` is valid. Then inspect the current process:

GDB TERMINAL - commands entered at the (gdb) prompt

```
p p->pid
p p->name
p p->state
p p->parent
p p->parent->pid
p/x p->pagetable
p p->sz
```

The process states are:

Value	State	Meaning
0	UNUSED	empty process-table slot
1	USED	slot is being prepared
2	SLEEPING	blocked on a channel
3	RUNNABLE	ready to run
4	RUNNING	currently executing on a hart
5	ZOMBIE	exited, waiting to be reaped

Run the helper again:

GDB TERMINAL - commands entered at the (gdb) prompt

```
procs
```

8.1 Inspect the state prepared for later labs

The V3.0 struct `proc` already contains fields that later labs will use:

GDB TERMINAL - commands entered at the (gdb) prompt

```
p/x p->trace_filter
p p->syscall_count
p p->ctxsw_count
p p->state_gen
p/x p->usysinfo
p p->usysinfo->version
p p->usysinfo->seq
p p->usysinfo->pid
```

Interpret the starter carefully:

- `trace_filter` begins at zero, and the Lab 2 control/recording logic is intentionally unfinished.
- `syscall_count` already increments at the syscall dispatch point.

- `ctxsw_count` and `state_gen` already change in the scheduler.
- `usysinfo` already points to a mapped physical page, and `version` is initialized to 1.
- The Lab 3 updater is still a TODO, so fields such as the shared page's `pid`, `ticks`, and `seq` are **not yet expected to track the live process correctly**.

Do not use later expected output as a baseline expectation

Before Lab 2 is implemented, `trace ls` is not expected to produce the final event stream. Before Lab 3 is implemented, `usitest` is not expected to report the final fast-getpid/torn-snapshot success output. Today's task is to inspect the hook points and existing mappings, not to make those tests pass.

9 Practice F - Locate the Future Hook Points

This final section deliberately stops *inside* the two later-lab hooks without asking you to implement them.

9.1 Lab 2 hook: `trace_record()`

Set a breakpoint:

GDB TERMINAL - commands entered at the (gdb) prompt

```
b trace_record
c
p num
p ret
p/x arg0
bt
```

The backtrace should show that `trace_record()` was called from `syscall()` after the handler returned. That placement gives the tracer both the original argument and the final return value.

Because the recorder is still a starter stub, ring state will not advance yet. You may inspect its initialized state while stopped inside `trace_record()`:

GDB TERMINAL - commands entered at the (gdb) prompt

```
p rings[0].head
p rings[0].count
p rings[0].dropped
```

Practice F1 - Why after the handler?

Why would a tracer placed *before* `syscalls[num]()` be unable to record the real system-call return value without additional work?

9.2 Lab 3 hook: `usysinfo_update()`

Delete or disable the trace breakpoint, then stop in the shared-page updater:

GDB TERMINAL - commands entered at the (gdb) prompt

```
b usysinfo_update
c
p p->pid
p p->syscall_count
```

```
p p->usysinfo->seq
list
```

In the current starter, the updater body is intentionally unfinished, so stepping through it does not yet perform the future odd/update/even sequence. The important observation today is *where* the kernel calls the updater and which live fields it will eventually mirror into the shared page.

Practice F2 - Why consistency becomes a problem

Suppose user code reads `pid`, `ticks`, and `syscall_count` from the shared page while the kernel is updating those fields. Why is mapping the page read-only not by itself enough to guarantee a consistent multi-field snapshot?

10 Final Concept Check

Discuss these with your partner or TA. Nothing is submitted.

1. At `syscall()`, where is the system-call number stored, and why is the trapframe the reliable place to inspect it?
2. What does `scause == 8` mean, and what does `usertrap()` do to the saved PC before dispatch?
3. While a process is inside the kernel handling a syscall, which page-table root did your `satp` comparison show?
4. Why can the kernel run with a process page table still active in Luit?
5. If process A and B have different roots, how do both reach the same kernel physical pages?
6. Why does the info page have `PTE_U=1` but kernel text has `PTE_U=0`?
7. Why must the info page have `PTE_W=0` in user mode?
8. What information is available to `trace_record()` only because it runs after the syscall handler returns?
9. Why might a later tracer prefer one ring per hart rather than one global ring?
10. Why does a multi-field shared information page need a consistency protocol in addition to page-table permissions?

Checkpoint: You are ready for the later labs if you can explain these without guessing

You should now be able to point to the exact live evidence for: the user-to-kernel trap, the syscall ABI, the active page-table root, a leaf PTE's permissions, the physical page behind a user VA, and the current process's kernel state.

11 Clean Exit and Troubleshooting

11.1 Clean exit

In GDB:

GDB TERMINAL - commands entered at the (gdb) prompt

```
delete breakpoints
quit
```

If QEMU remains running in Terminal 1, exit it with **Ctrl-A, then X**.

11.2 Common problems

Symptom	What to check
GDB does not show the Luit helper message	The project <code>.gdbinit</code> was not auto-loaded; check the safe-path setting.
GDB cannot connect to port 1234	Start <code>make qemu-gdb</code> first and make sure another QEMU from your account is not already using the port.
<code>p p->...</code> gives nonsense immediately at <code>syscall()</code>	Step once over <code>struct proc *p = myproc()</code> ; before using the local variable <code>p</code> .
QEMU console appears frozen	This is normal while GDB has stopped the virtual CPU at a breakpoint. Use <code>c</code> to resume.
A syscall breakpoint triggers repeatedly during boot	Inspect the process name, then continue; disable the breakpoint when you want to interact with the guest shell.
<code>walk(...)</code> returns zero	Make sure you are stopped in a valid process context and are walking <code>p->pagetable</code> ; the V3.0 info-page mapping is established when the process is allocated.
<code>trace ls</code> or <code>usitest</code> does not show later-lab success	Expected on the untouched starter: the Lab 2 recorder and Lab 3 updater are still TODOs.

12 One-Page GDB Quick Reference

Command	Purpose
<code>make qemu-gdb</code>	Start QEMU paused with the remote debugger on port 1234.
<code>gdb-multiarch kernel.elf</code>	Load kernel symbols and attach using the shipped <code>.gdbinit</code> .
<code>b syscall</code>	Stop at the one dispatch point every system call crosses.
<code>b usertrap</code>	Stop at user-mode trap handling.
<code>b scheduler</code>	Stop at the per-hart scheduling loop.
<code>c</code>	Continue execution until the next breakpoint.
<code>next / step</code>	Step over / into one source-level operation.
<code>bt</code>	Show the current call stack.
<code>list</code>	Show source around the current line.
<code>p expr</code>	Print a C expression.
<code>p/x expr</code>	Print an expression in hexadecimal.
<code>trapregs</code>	Print <code>scause</code> , <code>sepc</code> , <code>stval</code> , <code>sstatus</code> , and <code>satp</code> .
<code>procs</code>	Compact process-table display from the shipped helper.
<code>tf p->tf</code>	Print selected saved user registers from the current trapframe.
<code>walk(pt,va,0)</code>	Ask Luit to locate the PTE for a virtual address without allocating.

The debugger is not a separate theory topic. It is how you test whether your mental model of the operating system matches the machine.